

Module 06 | Foundations of Multi-Layer Perceptrons and Forward Propagation

My Solution to Practice Problems

Table of Contents

- Q01. What happens if we stack multiple linear layers without activation?
- Q02. What kind of decision boundary does an MLP create?
- Q03. What are the main components of a neural network?
- Q04. What is the difference between input layer, hidden layer, and output layer?
- Q05. What is forward propagation?

Q01. What happens if we stack multiple linear layers without activation?

Without (non-linear) activation functions, the output of a linear neuron is $\hat{y} \equiv \vec{W} \cdot \vec{x} + b$, where $\vec{W} \equiv [w_i]$ is the row vector of weights and $x \equiv [x_i]$ is the input sample, that is, the column vector of features values. So, without (non-linear) activation functions, the output of a linear layer is:

$$\vec{O}^{(l)} \equiv \vec{W}^{(l)} \vec{O}^{(l-1)} + \vec{B}^{(l)}$$

, where, with m denoting the number of input samples, $n^{(l)}$ denoting the width of Layer l , that is, the number of neurons in Layer l , and $\vec{O}^{(0)} \equiv \vec{X} \equiv \left[[x_{ik}]_{k=1}^{k=m} \right]_{i=1}^{i=n^{(0)}}$ denoting the row vector of the column input samples,

$\vec{O}^{(l)} \equiv \left[o_i \right]_{i=1}^{i=n^{(l)}}$ is the column vector of outputs of the neurons in this layer.

$\vec{W}^{(l)} \equiv \left[[w_{ij}]_{j=1}^{j=n^{(l-1)}} \right]_{i=1}^{i=n^{(l)}}$ is the column vector of the row weight vectors of the neurons in this layer.

$\vec{O}^{(l-1)} \equiv \left[o_j \right]_{j=1}^{j=n^{(l-1)}}$ is the column vector of outputs of the neurons in the previous layer.

$\vec{B}^{(l)} \equiv \left[b_i \right]_{i=1}^{i=n^{(l)}}$ is the column vector of biases of the neurons in this layer.

When stack a linear layer over another linear layer,

$$\begin{aligned}\vec{O}^{(1)} &\equiv \vec{W}^{(1)} \vec{O}^{(0)} + \vec{B}^{(1)} \\ &\equiv \vec{W}^{(1)} \vec{X} + \vec{B}^{(1)}\end{aligned}$$

$$\begin{aligned}\vec{O}^{(2)} &\equiv \vec{W}^{(2)} \vec{O}^{(1)} + \vec{B}^{(2)} \\ &\equiv \vec{W}^{(2)} \left(\vec{W}^{(1)} \vec{X} + \vec{B}^{(1)} \right) + \vec{B}^{(2)} \\ &\equiv \underline{\vec{W}^{(2)} \vec{W}^{(1)}} \vec{X} + \underline{\vec{W}^{(2)} \vec{B}^{(1)} + \vec{B}^{(2)}},\end{aligned}$$

which is still a linear transformation of $\vec{X}$,
as the underlined parts are constants.

More generally, for a n -layer stack of linear layers,

$$\vec{O}^{(l)} \equiv \underline{\vec{W}^{(l)} \vec{W}^{(l-1)} \dots \dots \dots \vec{W}^{(1)}} \vec{X} + \underline{\sum_{i=1}^{i=l} \left(\left(\prod_{j=i+1}^{j=l} \vec{W}^{(j)} \right) \vec{B}^{(i)} \right)}$$

Again, just a linear transformation, so replaceable by a single linear neuron. A stack of linear layers collapse into a single linear neuron. Without non-linear activation function, the depth of a network does not add any expressive power, still linear in capability.

Q02. What kind of decision boundary does an MLP create?

An **MLP (Multi-Layer Perceptron)** creates a **nonlinear decision boundary**.

- A single-layer perceptron creates only a **linear** decision boundary (a hyperplane).
- By stacking multiple layers with **nonlinear activation functions** (e.g., ReLU, tanh, sigmoid), an MLP can learn **complex, curved, and highly irregular** decision boundaries.
- The more hidden units and layers an MLP has (assuming sufficient data and proper training), the more intricate the decision boundary it can represent.

In short:

MLP = Flexible nonlinear decision boundaries.

Q03. What are the main components of a neural network?

1. Input Layer

- Receives the input features.
- One neuron per input feature (typically).

2. Hidden Layer(s)

- Perform computations and extract patterns.
- A network may have one or many hidden layers.

3. Neurons (Nodes)

- The basic computational units.
- Each neuron computes:

$$z = \mathbf{w}^\top \mathbf{x} + b$$

followed by an activation function:

$$a = f(z)$$

4. **Weights (w)**

- Determine the strength of connections between neurons.
- These are the primary parameters learned during training.

5. **Biases (b)**

- Allow neurons to shift their activation threshold/level.
- Improve the flexibility of the model.

6. **Activation Functions**

- Introduce nonlinearity.
- Common examples:
 - ReLU
 - Sigmoid
 - Tanh
 - Softmax (usually in the output layer for multi-class classification)

7. **Output Layer**

- Produces the final prediction.
- Structure depends on the task:
 - Regression: one linear neuron
 - Binary classification: one sigmoid neuron
 - Multiclass classification: multiple softmax neurons

8. **Loss Function**

- Measures how wrong the predictions are.
- Examples:
 - Mean Squared Error (MSE)
 - Binary Cross-Entropy
 - Categorical Cross-Entropy

9. **Optimizer**

- Updates the weights and biases to minimize the loss.
- Examples:
 - Gradient Descent
 - SGD
 - Adam
 - RMSprop

In One Sentence,

A neural network consists of **layers of neurons connected by weights and biases, using activation functions to learn complex patterns, trained by minimizing a loss function with an optimizer.**

Q04. What is the difference between input layer, hidden layer, and output layer?

Input Layer

- Receives the raw input data.
- Does **not** perform meaningful computation.
- Simply passes the input values to the first hidden layer.
- Number of neurons = number of input features.

Example:

If each image is represented by 784 pixel values, the input layer has **784 neurons**.

Hidden Layer

- Performs computations on the input.
- Learns intermediate features and patterns.
- Each neuron computes a weighted sum, adds a bias, and applies an activation function.
- A network may have one or many hidden layers.

Example:

A hidden layer might learn edges, shapes, textures, or higher-level concepts from an image.

Output Layer

- Produces the final prediction.
- Its structure depends on the task.

Examples:

- **Regression:** 1 linear neuron
- **Binary Classification:** 1 sigmoid neuron
- **Multiclass Classification:** One softmax neuron per class

Summary

Layer	Purpose	Learns?
Input Layer	Receives input features	No
Hidden Layer	Learns intermediate representations	Yes
Output Layer	Produces the final prediction	Yes

Q05. What is forward propagation?

Forward propagation is the process of passing input data through a neural network—from the **input layer**, through the **hidden layer(s)**, to the **output layer**—to produce a prediction.

At each neuron:

1. Compute the weighted sum:

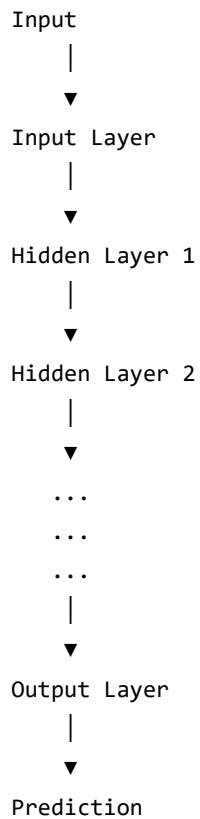
$$z = \mathbf{w}^\top \mathbf{x} + b$$

2. Apply an activation function:

$$a = f(z)$$

3. Pass the output to the next layer.

This process repeats until the network reaches the output layer.



Forward propagation is used to:

- Compute the network's prediction.
- Calculate the loss by comparing the prediction with the true target.
- Provide the values needed for **backpropagation**, which generates the gradients needed by an optimizer to update the model's parameters.

In short,

Forward propagation is the process of computing a neural network's output from its input using the current weights and biases.